

Approximate dictionary

Bloom filters can be seen as an example of approximate dictionary with error probability f .

- membership $x \in S$ with error prob. f

cost $\left[\begin{array}{l} \text{- uses } k = \frac{m}{n} \ln 2 \text{ hash functions} \Rightarrow \\ \quad O(k) \text{ time} \\ \text{- Space is } 1.44 \lg(1/f) n \end{array} \right. \quad n = |S|$

Bloom filters are widespread as of today

Q. Is this the best possible theoretically?

A. NO! We can improve both time and space

PLAN

- ① We prove a lower bound that $n \lg(1/p)$ bits at least, are required
- ② We see a matching upper bound (apart from lower order terms)
 - succinct RANK data structure
 - 1 hash function

The solution is static, i.e. it is expensive to add keys

LOWER BOUND ①

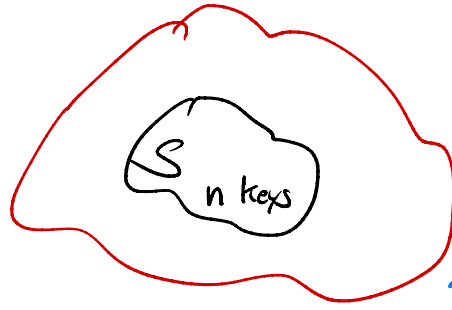
a) Folklore lower bound in information theory:

If we have a class of K objects, we need at least $\lg_2 K$ bits to distinguish among them

proof By contradiction, using less bits makes two objects in the class to be identified by the same bits by the pigeonhole principle

b) With Bloom Filters and, in general, with the approximate dictionary we want to store n keys out of a universe of m keys

a) $\Rightarrow$ we need at least $\lg \binom{m}{n}$ bits

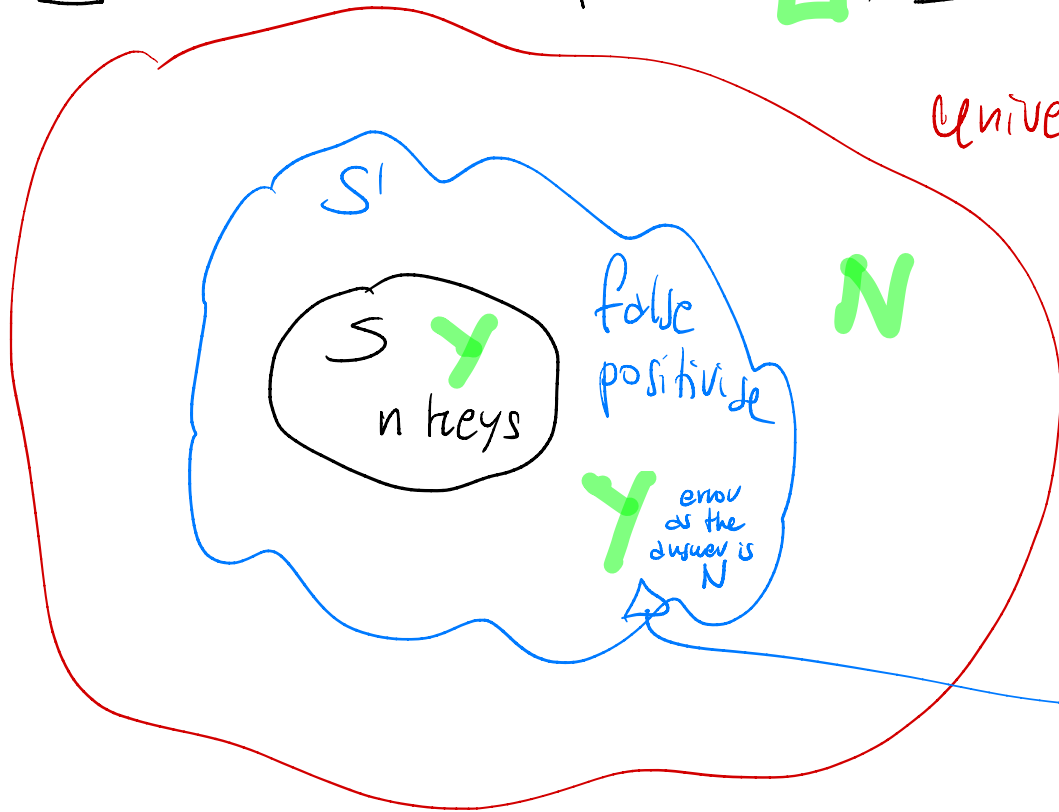


$m = |U|$ $n = |S|$
Universe of m keys

we have $K = \binom{m}{n}$ possible sets $S \subseteq U$

c | Let's consider the probability p of false positives

universe of m keys



$$S \subseteq S'$$

and $S' \setminus S = \text{false positives}$

$$|S' \setminus S| = p \cdot m$$

$x \in S \Rightarrow \text{BF says YES}$

$x \notin S \Rightarrow \text{BF says YES/NO} \begin{cases} \text{BF says YES for } S' \\ \text{BF says NO for Universe} \setminus S' \end{cases}$

← ERROR prob p

obs. An approximate dictionary with error prob. ϵ recognizes S' instead of S
 (whereas a classical dictionary recognizes just S but needs to store also S')

Recall instead that we do not store S' !

Our argument

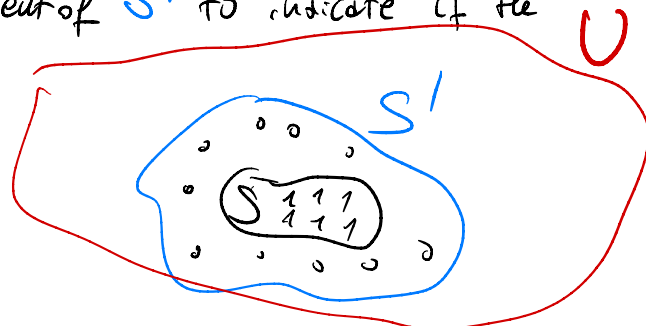
Let b the minimum number of bits needed to store an approximate dictionary for S
 (it is not possible to use less than b bits)

As we saw, these b bits describe $S' \supseteq S$

We can add further bits so that we can also represent S
 (for example, we use 1 bit for each element of S' to indicate if the element is also in S)

$$|S| = n$$

$$|S'| = |S| + \left\lceil \epsilon f m \right\rceil \leq n + \epsilon f m$$



Hence we need $\leq n + fm$ bits in addition to b bits
 and n of these bits are 1

$$b + \overbrace{11111}^{n+fm \text{ bits}} \sim b + \log_2 \binom{n+fm}{n}$$

$\nwarrow \nearrow$
 $n \text{ bits} = 1$

$\uparrow$
 approx
 dictionary
 S'

In this way, using at least $b + \log \binom{n+fm}{n}$ bits we can encode S'

However, we saw before that any encoding of S requires $\geq \log \binom{m}{n}$ bits in the worst case by the information-theory argument

this encodes S'

$$\geq b + \log \binom{n+fm}{n} \geq \log \binom{m}{n}$$

this encodes S

← this is the crucial inequality

$$b \geq \lg\binom{m}{n} - \lg\binom{n + \frac{f_m}{n}}{n} \sim n \lg \frac{m}{n} - n \lg \left(\frac{n + \frac{f_m}{n}}{n} \right) \sim n \lg \frac{m}{n} - n \lg \frac{f_m}{n} \quad \Rightarrow$$

$$\lg\left(\frac{a}{b}\right) \sim b \lg \frac{a}{b} \quad 1 + \frac{f_m}{n} \sim \frac{f_m}{n}$$

$$n \lg \left(\frac{\cancel{m}}{\cancel{n}} \cdot \frac{\cancel{n}}{\cancel{f_m}} \right) = n \lg \left(\frac{1}{f} \right)$$

bits required
per key



② SUCCINCT RANK DATA STRUCTURE

Input: a binary vector B of m bits where n bits are 1

→ (recall that IT says that $\log_2 \binom{m}{n}$ bits are required to store B
and that $\log_2 \binom{m}{n} \leq m$)

Output: rank data structure that "implicitly" encodes B
and supports constant-time queries

$$\text{rank}(B, i) = \#1\text{s in } B[1..i]$$

example: $B =$ 0 1 0 1 1 0 0 1 0 1 1 1 0 1
 1 2 3 4 5 6 7 8 9 10 11 12 13 14

$$m = 14$$

$$n = 9$$

$$\text{rank}(B, 6) = 3$$

$$\text{rank}(B, 7) = 3$$

$$\text{rank}(B, 8) = 4$$

Hence $B[i] = 1$ iff $\text{rank}(B, i-1) \neq \text{rank}(B, i)$
(by def. $\text{rank}(B, 0) = 0$)

Implementation

0	1	0	1	1	0	1	0	0	1
1	2	3	4	5	6	7	8	9	10

B

First idea

0	1	1	2	3	3	4	4	4	5
---	---	---	---	---	---	---	---	---	---

rank(B, i)

rank
← prefix sum

too much space: $|B| \cdot \lg |B| =$
m log m bits

Second idea: let's sample the prefix sums!

0	1	1	2	3	3	4	4	4	5	...
---	---	---	---	---	---	---	---	---	---	-----

$\frac{1}{2} \lg m$

space now reduces to $\frac{m}{\frac{1}{2} \lg m}$. $\lg m$ or $2m$ bits

sampled
prefx sums

query time is $O(\lg m) \rightarrow$ nearest sampled prefx sum
+ sum of $< \frac{1}{2} \lg m$ bits

Output time

this cause $\Theta(\lg m)$ cost

this is actually $O(1)$
using a table of size
 $2^{\frac{1}{2} \lg m} = O(\sqrt{m})$ bits

$h: U \rightarrow [r]$
'pr. = $\frac{1}{r} = f$